

IFLUX LoRA Cartoon Pipeline

ML/AI Independent Study Project

Christina Ross

NetID: Car476

Spring 2025

1. Project Overview

This project is a diffusion-based image generation pipeline built in Google Colab. The goal was to fine-tune a FLUX diffusion model using LoRA to generate consistent cartoon-style images and character poses from prompts and reference images. The main external inputs needed are the pre-trained LoRA weights and the training images. Everything else is handled inside the notebook pipeline.

The pipeline does two main things. First it generates cartoon-style images from text prompts or reference images using the FluxKontext model with the trained LoRA weights loaded. Second it has the full LoRA training workflow built in so the weights can be updated or retrained if needed.

2. Environment Setup

Everything runs in Google Colab using an A100 GPU with High RAM. To get started you go to Runtime > Change runtime type and select A100 with High RAM before running anything.

Dependencies installed at the top of the notebook:

```
!pip install -q diffusers transformers accelerate peft torch
huggingface_hub bitsandbytes torchvision
```

You also need to log into Hugging Face since FLUX.1-dev and FLUX.1-Kontext-dev are gated models that require access. That is handled with:

```
from huggingface_hub import login
login()
```

Google Drive is mounted to pull in the saved LoRA weights so you do not have to retrain from scratch every session:

```
from google.colab import drive
drive.mount('/content/drive')
```

3. Model

The pipeline uses FluxKontextPipeline from Hugging Face Diffusers. Specifically it loads FLUX.1-Kontext-dev because it supports image-to-image generation, meaning you can give it a reference image and a prompt and it will generate a new pose that matches the same character.

```
from diffusers import FluxKontextPipeline
pipe = FluxKontextPipeline.from_pretrained(
```

```

        'black-forest-labs/FLUX.1-Kontext-dev',
        torch_dtype=torch.bfloat16
    )
    pipe.enable_model_cpu_offload()

```

The notebook supports both text-to-image and image-to-image generation depending on the workflow being used.

`enable_model_cpu_offload()` is important here. It keeps parts of the model in RAM and only moves what is needed to the GPU at a time, which helps manage memory even on higher-end runtimes.

FLUX.1-schnell was also tried but commented out because the quality was not as good. FLUX.1-Kontext-dev gives better results even though it is a bigger model.

4. LoRA Training

4.1 What LoRA Does

LoRA stands for Low-Rank Adaptation. Instead of retraining the entire FLUX model which would require way more compute than a free or Pro Colab runtime can handle, LoRA adds a small set of trainable weights on top of the frozen base model. After training you get a `.safetensors` file that is only a few hundred MB and you load it on top of the base model to activate the style.

4.2 Training Data and Captions

The training images are the cartoon character PNGs. Each image needs a caption file so the model knows what style to associate with the images. The notebook auto-generates captions from the filenames. For example `dog_happy.png` becomes:

```

frmcrttoon style cartoon dog happy pose, flat design, thick outlines,
white background

```

The training images were already captioned but the original captions did not contain the trigger word `frmcrttoon`. Initially the trigger word was added through the instance prompt flag during training:

```

--instance_prompt='frmcrttoon style cartoon character'

```

This let the model associate the dataset with the trigger word without having to manually edit every caption file. It was faster but the tradeoff is the model learns the style more loosely instead of binding it tightly to the trigger word. Later in the project the caption generation script was updated so the trigger word gets written directly into every caption file for stronger style consistency. An example of what a caption looks like now:

4.3 Training Command

Training runs with `accelerate launch` using the DreamBooth LoRA script from Hugging Face:

```

!accelerate launch train_dreambooth_lora_flux.py \
    --pretrained_model_name_or_path='black-forest-labs/FLUX.1-dev' \
    --instance_data_dir='training_data' \
    --output_dir='lora_output' \
    --mixed_precision='bf16' \

```

```

--instance_prompt='frcrtoon style cartoon character' \
--resolution=512 \
--train_batch_size=1 \
--gradient_accumulation_steps=4 \
--learning_rate=1e-4 \
--max_train_steps=1000 \
--rank=4 \
--checkpointing_steps=500 \
--gradient_checkpointing \
--use_8bit_adam

```

A few things worth noting here. `gradient_checkpointing` reduces how much VRAM is used during training. `use_8bit_adam` does the same for the optimizer. Both are needed to stay within Colab's memory limits. Checkpoints save every 500 steps so if the runtime disconnects you can pick up from the last checkpoint instead of starting over.

4.4 Saving the Weights

Once training finishes the weights get saved to `lora_output/` and downloaded as a zip:

```

with zipfile.ZipFile('my_cartoon_lora.zip', 'w') as z:
    for f in os.listdir('lora_output'):
        z.write(f'lora_output/{f}')
files.download('my_cartoon_lora.zip')

```

The weights were zipped for optional download, but the primary backup method was saving directly to Google Drive. This was one of the biggest lessons from earlier in the project. If you only save locally to Colab and the runtime crashes, everything is gone. Saving to Drive means you only have to train once and can reload the weights in any future session.

5. Image Generation

Once the LoRA weights are loaded, the pipeline generates cartoon-style images from a text prompt or a reference image. You pass in a prompt describing what you want, optionally with a reference image, and the model outputs a new image in the trained style.

Example for a single pose:

```

input_image = load_image('dog_happy.png')
prompt = 'frcrtoon style cartoon dog running, flat design, thick black outlines'
result = pipe(image=input_image, prompt=prompt,
               num_inference_steps=28, guidance_scale=3.5).images[0]

```

The notebook also has a batch mode that generates multiple poses at once from a list. For example it generates a full 34-frame walk cycle by repeating an 8-pose sequence:

- Frame 1: left front leg forward, right back leg back
- Frame 2: legs beginning to cross, slight body dip
- Frame 3: both legs centered, lowest body position

- Frame 4: legs crossing past center, body rising
- ...repeating to fill 34 frames

Each frame saves sequentially into an output folder. The batch approach is way more efficient than generating one at a time because you set it up once and let it run.

6. Issues and Lessons Learned

6.1 Storage

The biggest issue early on was files not saving correctly. Since everything runs in Colab instead of locally, downloads that looked like they were going to my PC were actually not saving right. When the runtime disconnected everything was gone.

The fix was routing all outputs to Google Drive from the start. Checkpoints, weights, and generated images all save there automatically. This meant I could disconnect, close the tab, come back the next day and just remount Drive and continue. It also meant I was not wasting GPU time re-running training from scratch every session which matters a lot when Colab Pro has compute limits.

6.2 Style Consistency

Getting the model to consistently generate the same cartoon style was the other main challenge. In an earlier version of the training setup the trigger word `frmcrttoon` was only in the instance prompt and not in the individual caption files. Because of that the model learned the cartoon style loosely instead of tightly associating it with the trigger word.

This is why in the current notebook the caption generation script writes the trigger word into every single caption file. When `frmcrttoon` shows up in both the captions and the prompt at inference time the model has a much stronger association and the outputs are more consistent.

6.3 Model Offloading

Another issue that came up was related to model offloading. After enabling `enable_model_cpu_offload()` in the pipeline setup, manually calling `pipe.to('cuda')` afterward was causing warnings and was actually reducing the benefits of offloading. The fix was removing the manual CUDA transfer entirely and just letting Diffusers handle GPU placement automatically. Once that was removed the pipeline ran cleanly without the warnings.

6.4 Memory

The A100 with High RAM handles FLUX.1-Kontext-dev well, but FLUX is still a large model and memory management still matters. The three things that keep it running smoothly are `enable_model_cpu_offload()`, `gradient_checkpointing` during training, and `use_8bit_adam` for the optimizer. All three are in the notebook by default.

7. Results

The training process successfully produced a usable LoRA model. The results below show the three stages of training and how the outputs changed as the model learned the style.

Before Training

This is what the base FLUX model generates with no LoRA weights loaded. The prompt used was a generic cartoon character prompt with no trigger word. The output is a clean cartoon image but it has no connection to the trained style.

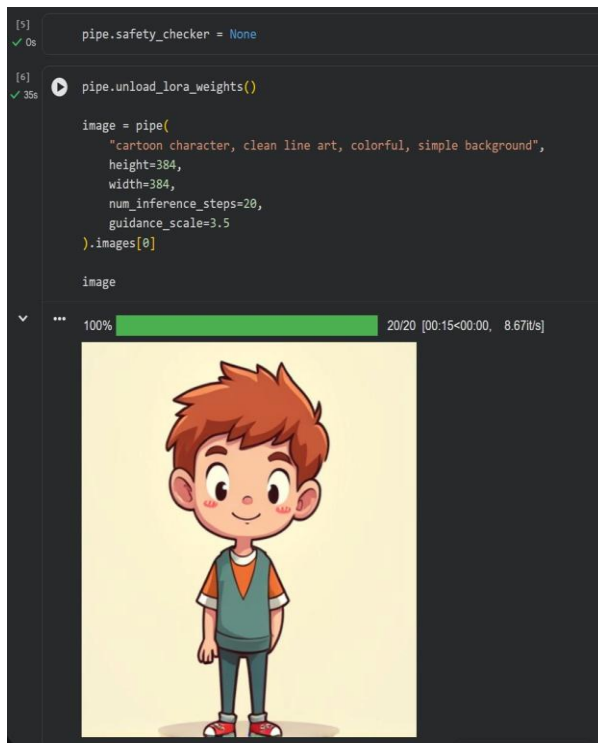


Figure 1: Base FLUX model output before any LoRA training. Generic cartoon character, no style binding.

With Some Training (Partial Weights)

This is with partial training where not all weights saved correctly. You can see the model is starting to pull toward a cartoon animal style which shows the training was having an effect, but the outputs are still inconsistent because the weights were incomplete.

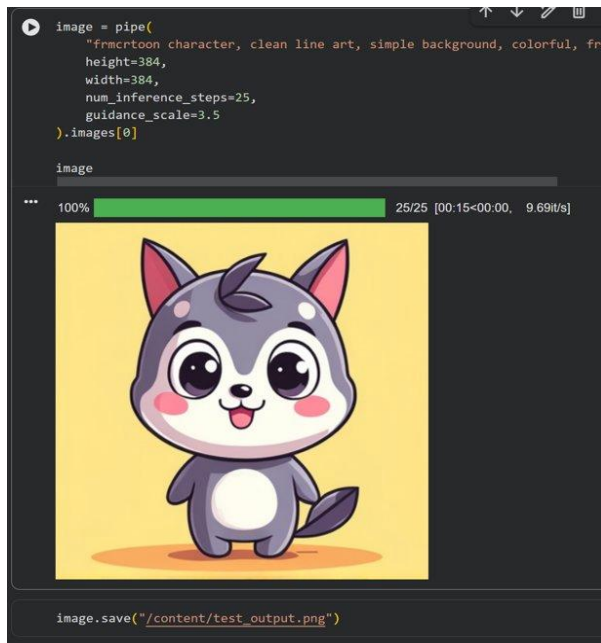


Figure 2: Output with partial LoRA weights. Style influence visible but inconsistent.

Full Training

With the fully trained weights loaded and frmcrttoon in the prompt the model generates cartoon-style character images that match the training data style. The style is noticeably more consistent and closer to the target aesthetic compared to the base model outputs.



Figure 3: Output with fully trained LoRA weights and frmcrttoon trigger word in prompt.

Overall the project was a success. The LoRA model learned the style and the trigger word association works as intended. For the scope of this project the results were successful and usable for cartoon image generation workflows. If I were to do this again I would look into a dedicated GPU environment with longer uninterrupted runtimes to get more training steps in without Colab session limits.

8. How I Would Improve This

The original goal of this project was to build a full video generator that I could prompt to produce animated cartoon videos. I did not get there because I ran out of time and the compute cost to do that properly was more than what was available to me on Colab.

What I built ended up being the image generation piece of that larger pipeline. If I were to continue this, the main improvement would be extending it into an actual video generator where you can type a prompt and get a short animated video back. That would mean adding a TTS component for voiceover, a way to sequence the generated images into frames, and assembling everything into a final video output. The image generation and LoRA training side already works, so that part would carry over.